```
def coremsg(smsg, icore):
    return ' ' + smsg + ' from core: ' + str(icore)

if '__main__' == __name__:

    osrvr = Server()
    ncpus = osrvr.get_ncpus()

    djobs = {}
    for i in xrange(0, ncpus):
        djobs[i] = osrvr.submit(coremsg, ("hello FLOSS", i))

    for i in xrange(0, ncpus):
        print djobs[i]()
```

In the above example, we first create the PP execution server object, *Server*. The constructor takes many parameters, but the defaults are good, so I won't provide any. By default, the number of worker sub-processes created (on the local machine) is the number of cores or processors in your machine—but you can create any number, not depending on *ncpus*. Parallel programming can't be simpler than this!

PP shines mainly when you have to split a lot of CPU-bound stuff (say, intensive number-crunching code) into multiple parallel parts. It's not as useful for I/O-bound code, of course. Run the following Python code to see PP's parallel computation in action:

```
#! /usr/bin/env python
# === parcompute.py ===
from pp import Server

def compute(istart, iend):
    isum = 0
    for i in xrange(istart, iend+1):
        isum += i**3 + 123456789*i**10 + i*23456789

    return isum

if '__main__' == __name__:

    osrvr = Server()
    ncpus = osrvr.get_ncpus()

    #total number of integers involved in the calculation
    uinum = 10000

    #number of samples per job
    uinumperjob = uinum / ncpus

    # extra samples for last job
    uiaddtlstjob = uinum % ncpus
    djobs = {}
    iend = 0
    istart = 0
    for i in xrange(0, ncpus):
        istart = i*uinumperjob + 1
        if ncpus-1 == i:
            iend = (i+1)*uinumperjob + uiaddtlstjob
        else:
            iend = (i+1)*uinumperjob

        djobs[i] = osrvr.submit(compute, (istart, iend))

    ics = 0
    for i in djobs:
        ics += djobs[i]()

    print ' workers: ' + str(ncpus)
    print ' parallel computed sum: ' + str(ics)
```

These PP examples just give you a basic feel for it; explore the more practical *examples* included in the examples subdirectory of the PP source. The source also includes documentation in the doc subdirectory. You have to run the *ppserver.py* utility, installed with the module, on remote computational nodes, if you want to parallelise your script on a computing cluster. In cluster mode, you can even set a secret key, so that the computational nodes only accept remote connections from an 'authenticated' source. There are no major differences between the multi-core/multi-processor and cluster modes of PP, except that you'd run *ppserver.py* instances on nodes, with various options, and pass a list of the *ppserver* nodes to the *Server* object. Explore more about the cluster mode in the PP documentation that's included with it.

As we have seen, though the CPython GIL constrains multi-core utilisation in Python programs, the modules discussed here will let your programs run at full throttle on SMP machines to extract the highest possible performance. END

References

- Wikipedia—Parallel Computing: *http://en.wikipedia.org/wiki/Parallel_computing*
- Online documentation for the multiprocessing module: http://docs.python.org/library/multiprocessing.html
- *Pprocess home page: http://www.boddie.org.uk/python/pprocess.html*
- Parallel Python home page: *http://www.parallelpython.com*

By: Ankur Kumar Sharma

The author is a software developer and researcher. He likes to play classic rock on his guitar, read self-help books, write, and explore interesting things in his spare time. He blogs at *www.richnusgeeks.com*